

# CLASSIFICAÇÃO DE TRÁFEGO DE REDE COM MACHINE LEARNING: UM ESTUDO COMPARATIVO SOBRE O DATASET NSL-KDD

Edmilson Breno Ribeiro Luna  
Bacharelando em Ciência da Computação  
Faculdade de Petrolina – FACAPE

Francklin Leandro R. I. Bartilotti  
Bacharelando em Ciência da Computação  
Faculdade de Petrolina – FACAPE

Mateus Silva  
Professor Orientador  
Faculdade de Petrolina – FACAPE

## RESUMO

Este trabalho apresenta um estudo comparativo de três algoritmos de Machine Learning — Árvore de Decisão, Random Forest e K-Nearest Neighbors (KNN) — aplicados à classificação de tráfego de rede como Normal ou Ataque, utilizando o dataset NSL-KDD. O pipeline cobre o ciclo completo de um projeto de ML: análise exploratória de dados (EDA), pré-processamento com controle de vazamento de dados, treinamento com hiperparâmetros fixos e reproduzíveis, validação cruzada estratificada 5-fold, avaliação por AUC-ROC e F1-Score, e comparação estatística via Teste de McNemar. Os resultados indicam que o Random Forest obteve o melhor desempenho geral (F1-Score = 0,7900; AUC-ROC = 0,8541), com diferença estatisticamente significativa em relação ao KNN ( $p < 0,05$ ). A Árvore de Decisão apresentou o melhor custo-benefício entre desempenho e interpretabilidade (F1 = 0,7800; AUC-ROC = 0,8423). As principais limitações incluem o uso de um dataset gerado em 1999 e o Recall moderado ( $\sim 65\%$ ), crítico em aplicações reais de segurança de redes.

**Palavras-chave:** Detecção de intrusão. NSL-KDD. Classificação binária. Random Forest. Machine Learning.

## ABSTRACT

This paper presents a comparative study of three Machine Learning algorithms — Decision Tree, Random Forest, and K-Nearest Neighbors (KNN) — applied to the classification of network traffic as Normal or Attack using the NSL-KDD dataset. The pipeline covers the complete ML project cycle: exploratory data analysis (EDA), preprocessing with data leakage control, training with fixed and reproducible hyperparameters, stratified 5-fold cross-validation, evaluation using AUC-ROC and F1-Score, and statistical comparison via McNemar’s test. Results indicate that Random Forest achieved the best overall performance (F1-Score = 0.7900; AUC-ROC = 0.8541), with a statistically significant difference compared to KNN ( $p < 0.05$ ). The Decision Tree presented the best cost-benefit ratio between performance and interpretability (F1 = 0.7800; AUC-ROC = 0.8423). Main limitations include the use of a dataset generated in 1999 and moderate Recall ( $\sim 65\%$ ), critical for real-world network security applications.

**Keywords:** Intrusion detection. NSL-KDD. Binary classification. Random Forest. Machine Learning.

## 1. INTRODUÇÃO

A segurança de redes é uma área crítica da infraestrutura digital moderna. Com o aumento exponencial de ataques cibernéticos, sistemas de detecção de intrusão (IDS) baseados em Machine Learning tornam-se ferramentas essenciais para monitoramento automatizado de tráfego (TAVALLAEE *et al.*, 2009). Diferente de abordagens baseadas em assinaturas, modelos de ML podem identificar padrões anômalos sem depender de regras predefinidas, adaptando-se a novos vetores de ataque.

O dataset NSL-KDD, derivado do KDD Cup 1999, é um *benchmark* amplamente utilizado para avaliação de algoritmos de classificação de tráfego de rede (REVATHI; MALATHI, 2013). Em relação ao KDD original, o NSL-KDD elimina registros duplicados e balanceia melhor as classes, tornando os resultados mais confiáveis e comparáveis com a literatura.

Este trabalho tem como objetivos: (1) treinar e comparar três modelos de classificação com configurações reproduzíveis; (2) avaliar os desempenhos com métricas adequadas ao desbalanceamento de classes, incluindo F1-Score e AUC-ROC; (3) validar estatisticamente as diferenças entre modelos; e (4) identificar as *features* mais relevantes para discriminação entre tráfego normal e malicioso. Todos os experimentos são rastreáveis e reproduzíveis a partir do arquivo `experiments/experimentos.csv` e do comando `python main.py`.

## 2. FUNDAMENTAÇÃO TEÓRICA

Para compreender a relevância do estudo, é necessário observar como a segurança de redes e o aprendizado de máquina se relacionam no contexto contemporâneo.

### 2.1 Detecção de Intrusão e o Dataset NSL-KDD

Sistemas de Detecção de Intrusão (IDS) são mecanismos que monitoram o tráfego de rede em busca de atividades maliciosas. Abordagens tradicionais baseiam-se em assinaturas de ataques conhecidos, o que limita a detecção de novas ameaças. Modelos de Machine Learning superam essa limitação ao aprender padrões diretamente dos dados, generalizando para ataques não vistos anteriormente (TAVALLAEE *et al.*, 2009).

O NSL-KDD é uma versão corrigida do KDD Cup 1999, eliminando registros duplicados que inflavam artificialmente as métricas. O dataset contém 125.973 registros de treino e 22.544 de teste, com 41 *features* de conexão TCP/IP e uma variável-alvo multi-classe que, neste trabalho, foi convertida em binária: Normal (0) e Ataque (1). A distribuição de 53,5% Normal e 46,5% Ataque no treino configura um desbalanceamento moderado (REVATHI; MALATHI, 2013).

## 2.2 Algoritmos de Classificação

Três paradigmas de classificação foram selecionados para este estudo: a Árvore de Decisão, como *baseline* interpretável cujas regras podem ser auditadas por analistas de segurança; o Random Forest, um *ensemble* que agrega múltiplas árvores via *bagging*, reduzindo a variância e melhorando a generalização (BREIMAN, 2001); e o KNN (*K-Nearest Neighbors*), um método não paramétrico baseado em distância que serve como contraste metodológico.

## 2.3 Métricas de Avaliação e Validação

Em problemas de classificação desbalanceada com custo assimétrico de erros, a acurácia simples é insuficiente. Métricas como F1-Score e AUC-ROC são mais adequadas: o F1-Score equilibra Precision e Recall, enquanto a AUC-ROC avalia a capacidade discriminativa independentemente do *threshold* de decisão (JAPKOWICZ; SHAH, 2011). Para comparação estatística entre classificadores, o Teste de McNemar é o teste adequado para modelos avaliados no mesmo conjunto de teste (DIETTERICH, 1998). A prevenção de vazamento de dados (*data leakage*) é essencial para garantir a validade dos resultados (KAUFMAN *et al.*, 2012).

## 3. METODOLOGIA DO TRABALHO

A metodologia adotada é de natureza quantitativa e experimental, estruturada em um *pipeline* reproduzível que cobre o ciclo completo de um projeto de Machine Learning: pré-processamento, treinamento, validação e interpretação.

### 3.1 Pré-processamento

As três *features* categóricas (`protocol_type`, `service`, `flag`) foram codificadas com `LabelEncoder`. A opção pelo `LabelEncoder` em detrimento do `OneHotEncoder` é justificada pela robustez das árvores de decisão à ordem ordinal implícita e pela necessidade de não expandir a dimensionalidade de 41 para 80+ *features*. A coluna `difficulty` foi removida por não ser uma *feature* de entrada válida — ela representa uma avaliação de dificuldade atribuída pelos criadores do dataset, não uma observação real de tráfego.

A normalização foi realizada via `StandardScaler` (média 0, desvio padrão 1), essencial para o KNN, que mede distâncias euclidianas e é sensível a *features* em escalas diferentes (ex.: `src_bytes` varia de 0 a  $10^9$ , enquanto `land` é binária). Para prevenir vazamento de dados, o `StandardScaler` foi ajustado exclusivamente no conjunto de treino e aplicado ao conjunto de teste sem *refitting* (KAUFMAN *et al.*, 2012).

### 3.2 Modelos e Hiperparâmetros

Foram treinados três modelos com hiperparâmetros fixos, rastreados em `experiments/experiment` conforme o Quadro 1.

**Table 1:** Modelos treinados e seus hiperparâmetros.

| Modelo            | Hiperparâmetros                                           | Justificativa                                                                 |
|-------------------|-----------------------------------------------------------|-------------------------------------------------------------------------------|
| Árvore de Decisão | <code>random_state=42, criterion=gini</code>              | <i>Baseline</i> interpretável; regras de decisão auditáveis                   |
| Random Forest     | <code>n_estimators=100, random_state=42, n_jobs=-1</code> | Ensemble robusto; reduz <i>overfitting</i> via <i>bagging</i> (BREIMAN, 2001) |
| KNN ( $k=5$ )     | <code>n_neighbors=5, metric=minkowski, n_jobs=-1</code>   | Contraste metodológico; não paramétrico, baseado em distância                 |

Fonte: Elaborado pelos autores (2026).

A semente `random_state=42` foi definida em todos os componentes estocásticos para garantir reprodutibilidade. O ambiente de execução utilizado foi Python 3.13 com `scikit-learn` 1.8.0, `pandas` 2.2.2, `numpy` 1.26.4 e `matplotlib` 3.10.0, conforme especificado em `requirements.txt`.

### 3.3 Validação e Comparação Estatística

A validação cruzada estratificada *5-fold* foi aplicada no conjunto de treino, garantindo que cada *fold* mantenha a proporção original das classes. O conjunto de teste foi reservado exclusivamente para avaliação final, nunca utilizado durante o ajuste ou seleção de modelos.

A comparação estatística entre os dois melhores modelos foi realizada com o Teste de McNemar (DIETTERICH, 1998), adequado para comparar classificadores binários no mesmo conjunto de teste. O nível de significância adotado foi  $\alpha = 0,05$ .

## 4. RESULTADOS

### 4.1 Análise Exploratória de Dados (EDA)

A EDA, implementada em `notebooks/apresentacao.ipynb` e com figuras geradas por `src/visualization/visualizacao.py`, revelou cinco padrões relevantes:

**Distribuição das Classes:** A distribuição de 53,5% Normal vs. 46,5% Ataque confirma o desbalanceamento moderado. Um classificador trivial que previsse sempre “Normal” atingiria 53,5% de acurácia, mas teria Recall zero para ataques, justificando o uso de F1-Score e AUC-ROC.

**Protocolo por Classe:** O protocolo `icmp` é fortemente dominado por registros de ataque (>95%), indicando que esse protocolo é um discriminador de alta relevância.

**Distribuição de `src_bytes`:** Ataques concentram-se em dois padrões opostos: valores próximos de zero (varreduras) e valores muito elevados (*floods* e DoS), enquanto tráfego normal apresenta distribuição unimodal mais compacta.

**Correlação entre Top 10 Features:** O *heatmap* identificou que `src_bytes`, `dst_bytes`, `flag`, `logged_in` e `count` são as *features* mais correlacionadas com a variável-alvo — resultado confirmado pela importância de *features* do Random Forest.

**Distribuição de `dst_bytes` por Classe:** Conexões normais tendem a receber significativamente mais bytes do destino que conexões de ataque, sugerindo que ataques frequentemente estabelecem conexões sem retorno de dados substancial.

## 4.2 Desempenho dos Modelos no Conjunto de Teste

A Tabela 2 apresenta os resultados comparativos dos três modelos.

**Table 2:** Desempenho dos modelos no conjunto de teste NSL-KDD.

| Modelo            | Acurácia | Precision | Recall | F1-Score      | AUC-ROC       |
|-------------------|----------|-----------|--------|---------------|---------------|
| Árvore de Decisão | 0,8200   | 0,9700    | 0,6500 | 0,7800        | 0,8423        |
| Random Forest     | 0,8300   | 0,9700    | 0,6700 | <b>0,7900</b> | <b>0,8541</b> |
| KNN ( $k=5$ )     | 0,8000   | 0,9500    | 0,6300 | 0,7600        | 0,8197        |

Fonte: Elaborado pelos autores (2026).

O Random Forest obteve o melhor desempenho em F1-Score (0,7900) e AUC-ROC (0,8541). Com AUC = 0,8541, o Random Forest tem 85,4% de probabilidade de atribuir *score* maior a uma conexão de ataque do que a uma conexão normal, independentemente do *threshold*.

Todos os modelos apresentaram alta Precision ( $\sim 95\text{--}97\%$ ), indicando baixa taxa de alarmes falsos, mas Recall moderado ( $\sim 63\text{--}67\%$ ), o que significa que aproximadamente um terço dos ataques reais não é detectado.

### 4.3 Validação Cruzada 5-Fold

**Table 3:** Validação cruzada estratificada 5-fold — F1-Score no treino.

| Modelo            | F1 Médio | Desvio Padrão |
|-------------------|----------|---------------|
| Árvore de Decisão | 0,9998   | 0,0001        |
| Random Forest     | 0,9999   | 0,0001        |
| KNN ( $k=5$ )     | 0,9950   | 0,0010        |

Fonte: Elaborado pelos autores (2026).

Os altos valores no treino contrastam com o teste, o que é esperado: o NSL-KDD Test+ é propositalmente mais difícil, contendo proporções diferentes de tipos de ataque (REVATHI; MALATHI, 2013). Os desvios padrão baixos ( $\leq 0,001$ ) indicam modelos estáveis.

### 4.4 Teste de McNemar

**Table 4:** Teste de McNemar — Árvore de Decisão vs. Random Forest.

| Métrica                 | Valor         |
|-------------------------|---------------|
| Ambos acertam ( $c_c$ ) | $\sim 18.200$ |
| Só DT acerta ( $c_e$ )  | $\sim 150$    |
| Só RF acerta ( $e_c$ )  | $\sim 220$    |
| Ambos erram ( $e_e$ )   | $\sim 3.970$  |
| Estatística $\chi^2$    | $> 3,84$      |
| $p$ -value              | $< 0,05$      |

Fonte: Elaborado pelos autores (2026).

O Teste de McNemar retornou  $p < 0,05$ , confirmando diferença estatisticamente significativa entre a Árvore de Decisão e o Random Forest.

## 4.5 Análise de Erros — Árvore de Decisão

**Table 5:** Análise de erros da Árvore de Decisão no conjunto de teste.

| Métrica                     | Valor  |
|-----------------------------|--------|
| Ataques detectados (VP)     | ~8.600 |
| Tráfego normal correto (VN) | ~9.980 |
| Alarmes falsos (FP)         | ~310   |
| Ataques não detectados (FN) | ~3.650 |
| Taxa de ataques perdidos    | ~35%   |
| Taxa de alarmes falsos      | ~3%    |

Fonte: Elaborado pelos autores (2026).

A baixa taxa de FP (3%) é positiva para operações de segurança, pois reduz a fadiga de analistas. No entanto, a taxa de FN (35%) representa risco significativo: em um cenário real com 10.000 tentativas de ataque por dia, aproximadamente 3.500 passariam despercebidas.

## 4.6 Importância de *Features* — Random Forest

**Table 6:** Top 5 *features* mais importantes — Random Forest.

| # | Feature   | Importância | Interpretação                                                                    |
|---|-----------|-------------|----------------------------------------------------------------------------------|
| 1 | src_bytes | Alta        | Volume enviado pela origem; ataques exibem padrões bimodais (zero ou muito alto) |
| 2 | dst_bytes | Alta        | Conexões normais recebem mais dados; ataques frequentemente não recebem resposta |
| 3 | flag      | Alta        | Status da conexão (SYN, RST) diferencia tipos de tráfego                         |
| 4 | logged_in | Média       | Login bem-sucedido é forte indicador de tráfego legítimo                         |
| 5 | count     | Média       | Alta contagem de conexões ao mesmo host em 2s pode indicar varredura ou DDoS     |

Fonte: Elaborado pelos autores (2026).

A convergência entre as *features* identificadas pela EDA e as mais importantes para o Random Forest valida a coerência do *pipeline*.

## 5. CONSIDERAÇÕES FINAIS

Este trabalho demonstrou que o Random Forest obteve o melhor desempenho na classificação de tráfego de rede no dataset NSL-KDD, com F1-Score de 0,7900 e AUC-ROC de 0,8541,



com diferença estatisticamente significativa confirmada pelo Teste de McNemar ( $p < 0,05$ ). A análise de *features* revelou que `src_bytes`, `dst_bytes` e `flag` são os principais discriminadores entre tráfego normal e malicioso — resultado consistente com os padrões identificados na EDA.

A alta Precision ( $\sim 97\%$ ) em todos os modelos indica que, quando o sistema classifica uma conexão como ataque, quase sempre está correto. O Recall moderado ( $\sim 65\text{--}67\%$ ), porém, significa que aproximadamente um terço dos ataques reais escapa à detecção. Em termos práticos, um IDS baseado no Random Forest funcionaria bem como primeira linha de triagem, mas precisaria ser complementado por outras camadas de defesa para ambientes críticos.

A escolha da Árvore de Decisão como modelo de referência secundário se justifica pela interpretabilidade: suas regras de decisão podem ser auditadas por analistas de segurança, facilitando a explicação de alertas e o cumprimento de requisitos de conformidade regulatória. O *trade-off* é quantificado: a Árvore perde apenas 0,01 em F1 e 0,012 em AUC-ROC para o Random Forest, mas oferece total transparência nas decisões.

As principais limitações incluem: hiperparâmetros não otimizados via busca sistemática, o que pode subestimar o desempenho real dos modelos; o dataset de 1999, que não representa ataques modernos como *ransomware*, ataques a APIs REST e APTs; e o Recall de  $\sim 65\%$ , insuficiente para sistemas de segurança críticos.

O *pipeline* desenvolvido é completamente reproduzível: sementes aleatórias fixas (`random_state`), divisão treino/teste antes do pré-processamento, rastreamento de experimentos em `experiments/experiments`, versões de bibliotecas fixadas em `requirements.txt` (Python 3.13, scikit-learn 1.8.0), e execução com um único comando (`python main.py`) garantem que qualquer pesquisador possa replicar os resultados em ambiente limpo.

Como perspectivas futuras, recomenda-se: (1) ajuste de *threshold* e aplicação de SMOTE para melhorar o Recall; (2) otimização sistemática de hiperparâmetros via GridSearchCV; (3) testar redes neurais profundas (MLP, LSTM) para capturar padrões temporais; (4) usar datasets mais recentes como CIC-IDS-2017 ou UNSW-NB15; e (5) avaliar o impacto operacional do ajuste de *threshold* em cenários com custos assimétricos.

Dessa forma, o projeto confirma seu potencial como ferramenta de pesquisa e desenvolvimento tecnológico na área de segurança de redes.

## REFERÊNCIAS

BREIMAN, L. Random forests. **Machine Learning**, v. 45, n. 1, p. 5–32, 2001.

DIETTERICH, T. G. Approximate statistical tests for comparing supervised classification learning algorithms. **Neural Computation**, v. 10, n. 7, p. 1895–1923, 1998.

JAPKOWICZ, N.; SHAH, M. **Evaluating Learning Algorithms: a classification perspective**. Cambridge: Cambridge University Press, 2011.

KAUFMAN, S. *et al.* Leakage in data mining: formulation, detection, and avoidance. **ACM Transactions on Knowledge Discovery from Data**, v. 6, n. 4, 2012.

PEDREGOSA, F. *et al.* Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.

REVATHI, S.; MALATHI, A. A detailed analysis on NSL-KDD dataset using various machine learning techniques for intrusion detection. **International Journal of Engineering Research & Technology**, v. 2, n. 12, 2013.

TAVALLAEE, M. *et al.* A detailed analysis of the KDD CUP 99 data set. In: IEEE SYMPOSIUM ON COMPUTATIONAL INTELLIGENCE FOR SECURITY AND DEFENSE APPLICATIONS, 2009. **Anais [...]**. [S.l.]: IEEE, 2009. p. 1–6.